They do this by mixing the code with the specification definitions, as is done in RSpec, but by also providing support for much richer BDD language structures.

There aren't many unit-testing BDD tools that support these features yet, but there are a few. In Scala, for example, Specs2 is one such tool (http://etorreborre.github.io/specs2/). And if you're working with Java or Groovy, you can use Spock (https://code.google.com/p/spock/). Both tools can be used very effectively to test Java code. Let's take a closer look at Spock.

Spock is a powerful and very expressive BDD unit-testing tool that lets you write clean and readable executable specifications in Groovy. Here's a simple example:

All Spock specifications extend the Specification class.

```
class WhenManagingFrequentFlyerMembers extends Specification {

    def "a new frequent flyer should have Bronze status"() {
        given:
          def member = FrequentFlyer.withFrequentFlyerNumber("12345678").
                                     named("Joe","Bloggs")
        when:
            def status = member.status
        then:
            status == FrequentFlyerStatus.BRONZE
    }
```

❶ **Given**

❷ **When**

❸ **Then**

Notice how this example is structured. The specification is organized into clear *Given* ❶, *When* ❷, and *Then* ❸ sections. Of course, Spock, like Gherkin, gives you a fair bit of flexibility in how you organize your "Given ... When ... Then" blocks, but they're nevertheless present and clearly marked.

Spock also makes example-driven testing very easy.

```
def "should upgrade status when enough status points are acquired"() {
    given: "a frequent flyer member with some points"
        def member = FrequentFlyer.withFrequentFlyerNumber("12345678").
                                   named("Joe","Bloggs").
                                   withStatusPoints(initialPoints).
                                   withStatus(initialStatus)
    when: "he earns some extra points on a flight"
        member.earns(extraPoints).statusPoints()
    then: "he may or may not be upgraded to a new status"
        member.getStatus() == expectedStatus
    where:
        initialStatus | initialPoints | extraPoints | expectedStatus
        BRONZE        | 0             | 299         | BRONZE
        BRONZE        | 0             | 300         | SILVER
        SILVER        | 0             | 699         | SILVER
        SILVER        | 0             | 700         | GOLD
        GOLD          | 0             | 1499        | GOLD
        GOLD          | 0             | 1500        | PLATINUM
}
```

❶ **Precondition**

Action ❷

❸ **Expected outcome**

❹ **Examples**

This specification, using a format that's quite similar to the Gherkin equivalent, feeds test data in from the examples table ❹. The column headers are injected into the *Given* ❶, *When* ❷, and *Then* ❸ steps, where they're used to do the actual test.